

Fraud Detection in Financial Transactions

Final Project Report

Introduction to Artificial Intelligence Project

EARIN – Summer Semester 2026

Project Topic: Fraud Detection

Dataset: PaySim Synthetic Financial Dataset

Final Model: Random Forest Classifier

Prepared by: Tharun Pranav Senthil Kumar
Esin Kutlu

Instructor: Kowaleczko Paweł

Project Aim

The aim of this project is to build and evaluate a machine learning model that can detect fraudulent financial transactions using transaction details and account balance information.

June 9, 2026

Contents

Abstract 3

1 Introduction 4

2 Dataset Description 4

2.1 Dataset Overview 5

2.2 Main Dataset Attributes 5

3 Exploratory Data Analysis 6

3.1 Fraud Distribution 6

3.2 Transaction Type Analysis 6

3.3 Transaction Amount Distribution 7

4 Data Preprocessing and Feature Engineering 8

4.1 Feature Engineering 8

4.2 Categorical Encoding 8

4.3 Removed Columns 8

4.4 Train-Test Split 9

5 Implemented Models 9

5.1 Logistic Regression Baseline 9

5.2 Random Forest Model 9

6 Midterm Model Comparison 10

7 Final Model Selection 10

8 Final Random Forest Results 11

8.1 Confusion Matrix 11

8.2 Final Evaluation Metrics 12

8.3 ROC Curve 13

8.4	Precision-Recall Curve	14
9	Feature Importance Analysis	14
9.1	Main Observations from Feature Importance	15
10	Discussion	15
10.1	Important Observations	16
10.2	Why Accuracy Alone Is Not Enough	16
10.3	Business Meaning	16
11	Handling Class Imbalance	16
11.1	Class Weighting	17
11.2	Stratified Sampling	17
12	Implementation Details	17
12.1	Libraries Used	17
12.2	Code Structure	17
12.3	How to Run the Code	19
12.4	Generated Output Files	19
13	Challenges Faced	19
14	Conclusion	20
15	References	20

Abstract

Fraud detection is an important application of artificial intelligence in financial systems. Modern payment systems process a large number of transactions every day, so manual fraud checking is not practical. Machine learning can help by automatically identifying transactions that look suspicious.

This project uses the PaySim synthetic financial dataset to detect fraudulent transactions. The task is treated as a binary classification problem, where each transaction is classified as either normal or fraudulent. During the midterm phase, two models were implemented and compared: Logistic Regression and Random Forest. In the final phase, Random Forest was selected as the main model because it gave better results, especially in reducing missed fraud cases and false alarms.

The final solution includes data exploration, preprocessing, feature engineering, model training, model evaluation, visualizations, and result discussion. The Random Forest model achieved a ROC-AUC score of **0.9993**. It detected **1,624** fraud cases out of **1,643** fraud cases in the test set and missed only **19** fraud transactions.

1 Introduction

Fraud detection is a major task in financial systems. Online banking, mobile payments, and digital wallets generate millions of transactions. A very small number of these transactions may be fraudulent, but they can still cause large financial losses.

The goal of this project is to create a machine learning model that predicts whether a transaction is fraudulent or legitimate. The prediction is based on transaction details such as transaction type, amount, sender balance, and receiver balance.

The task is a supervised binary classification problem:

$$f(x) = \begin{cases} 1, & \text{fraudulent transaction} \\ 0, & \text{legitimate transaction} \end{cases}$$

The project was completed in stages. The midterm phase focused on dataset exploration, initial preprocessing, and model comparison. The final phase focuses on completing the solution, selecting the final model, improving visualizations, and presenting the final results in a clear way.

Final Project Direction

Logistic Regression is kept as a baseline model from the midterm phase. Random Forest is selected as the final model because it gives better fraud detection performance and a lower number of false positives.

2 Dataset Description

The project uses the **PaySim Synthetic Financial Dataset**, available on Kaggle:

<https://www.kaggle.com/datasets/ealaxi/paysim1>

The dataset simulates mobile money transactions. It contains both normal and fraudulent transactions. It also includes useful information such as transaction type, transaction amount, account balances, and the fraud label.

2.1 Dataset Overview

Table 1: PaySim dataset overview

Property	Value
Total transactions	6,362,620
Normal transactions	6,354,407
Fraudulent transactions	8,213
Original number of features	11
Fraud ratio	0.129%
Machine learning task	Binary classification

The dataset is highly imbalanced. Fraud transactions make up only 0.129% of all transactions. This means that accuracy alone is not enough to evaluate the model. A model can have high accuracy even if it misses many fraud cases.

2.2 Main Dataset Attributes

Table 2: Main dataset attributes

Attribute	Description
step	Time step of the transaction.
type	Transaction type, such as PAYMENT, TRANSFER, CASH.OUT, DEBIT, or CASH.IN.
amount	Amount of money transferred in the transaction.
nameOrig	Identifier of the sender account.
oldbalanceOrig	Sender balance before the transaction.
newbalanceOrig	Sender balance after the transaction.
nameDest	Identifier of the receiver account.
oldbalanceDest	Receiver balance before the transaction.
newbalanceDest	Receiver balance after the transaction.
isFraud	Target column. Value 1 means fraud and value 0 means normal.
isFlaggedFraud	System flag for suspicious large transfers.

3 Exploratory Data Analysis

Exploratory data analysis was performed to understand the structure of the dataset before training the model. The main focus was on class imbalance, transaction types, and transaction amount distribution.

3.1 Fraud Distribution

A normal count plot is not clear for this dataset because the fraud class is very small compared with the normal class. Therefore, the final report uses a log-scale plot.

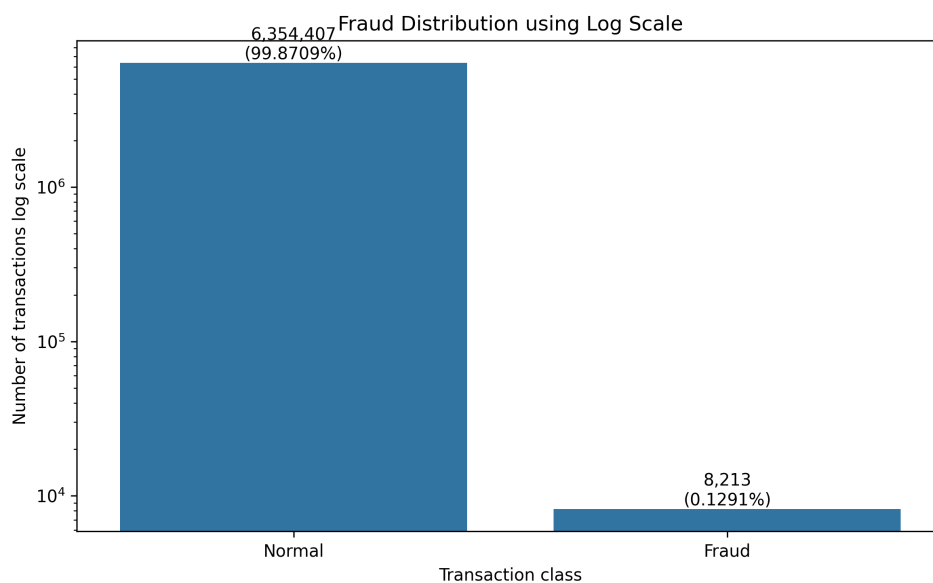


Figure 1: Fraud distribution using log scale. This makes the rare fraud class visible.

The plot confirms that the dataset is strongly imbalanced. There are more than 6.3 million normal transactions and only 8,213 fraud transactions. This imbalance affects both training and evaluation.

3.2 Transaction Type Analysis

Transaction type is important because fraud is not equally distributed across all transaction types.

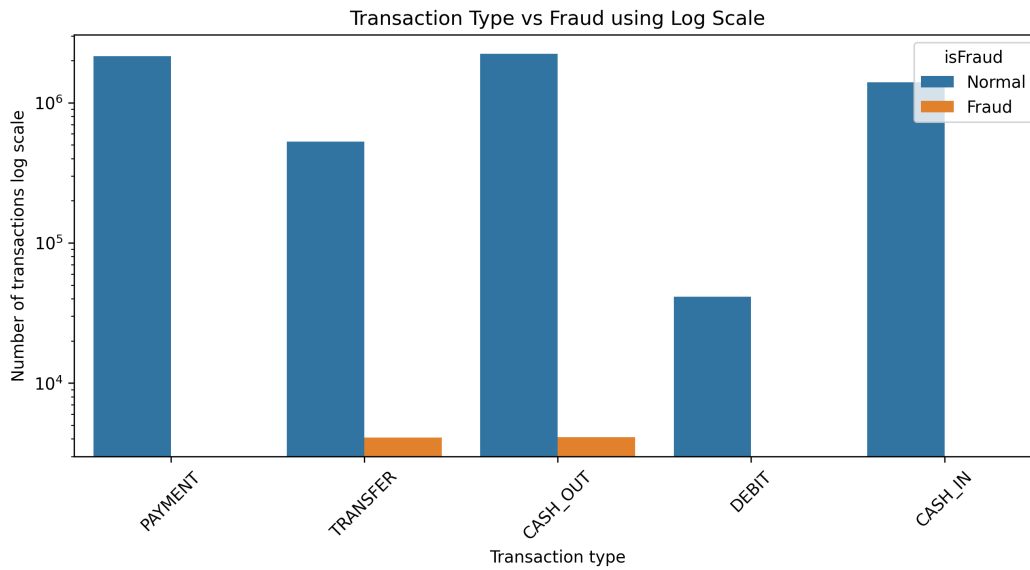


Figure 2: Transaction type versus fraud using log scale. Fraud cases are mainly visible in TRANSFER and CASH_OUT transactions.

The analysis shows that fraud mainly appears in TRANSFER and CASH_OUT transactions. This makes transaction type a useful feature for prediction.

3.3 Transaction Amount Distribution

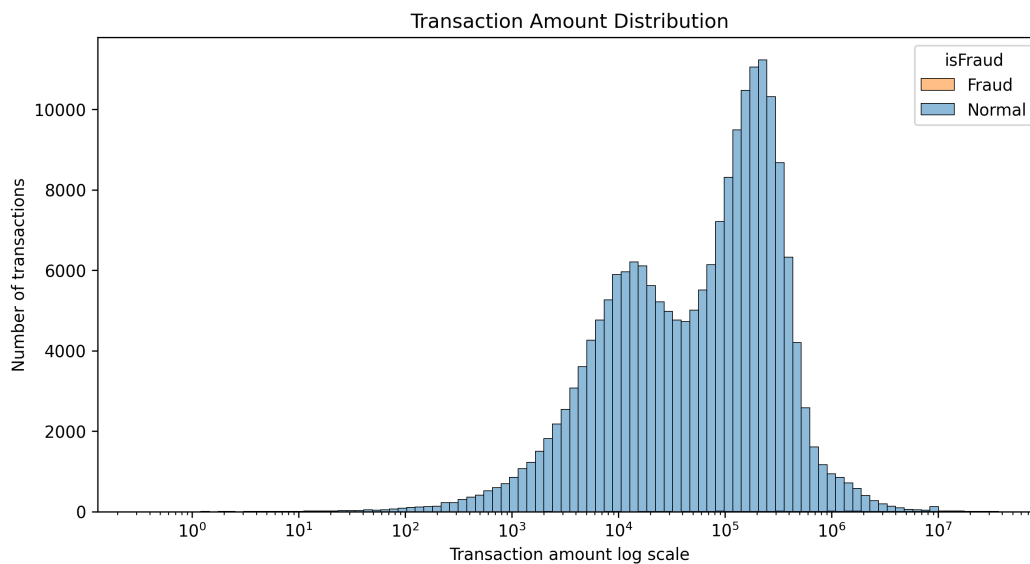


Figure 3: Transaction amount distribution on logarithmic scale.

Transaction amounts cover a large range of values. The logarithmic scale makes the distribution easier to view. Amount is useful, but it is not enough alone because normal and fraud transactions may overlap in amount.

4 Data Preprocessing and Feature Engineering

The preprocessing steps were kept simple and reproducible.

4.1 Feature Engineering

Two balance-based features were created:

$$balanceOrigDiff = oldbalanceOrg - newbalanceOrig$$

$$balanceDestDiff = newbalanceDest - oldbalanceDest$$

Table 3: Engineered features

Feature	Meaning
balanceOrigDiff	Amount removed from the sender account.
balanceDestDiff	Amount added to the receiver account.

These features help the model understand how the transaction changed the account balances. This is important because fraud transactions often create unusual balance changes.

4.2 Categorical Encoding

The `type` column contains text values. It was converted into numerical columns using one-hot encoding. This allows the model to use transaction types during training.

4.3 Removed Columns

Table 4: Removed columns

Column	Reason for removal
nameOrig	Account identifier. It does not provide useful general patterns.
nameDest	Account identifier. It does not provide useful general patterns.
isFlaggedFraud	System-generated flag. It was removed to avoid relying on an existing rule.

4.4 Train-Test Split

The dataset was split into training and testing data.

Table 5: Train-test split settings

Setting	Value
Training data	80%
Testing data	20%
Random state	42
Stratified split	Yes

Stratified splitting was used so that the fraud ratio remains similar in both training and testing data.

5 Implemented Models

Two models were used during the project. Logistic Regression was used as a baseline model, and Random Forest was used as the main model.

5.1 Logistic Regression Baseline

Logistic Regression was implemented during the midterm phase. It is simple, fast, and easy to understand. It was used as a baseline to compare with the main model.

The model used `class_weight='balanced'` to handle class imbalance.

$$P(y = 1|x) = \frac{1}{1 + e^{-(w^T x + b)}}$$

The Logistic Regression model achieved a high ROC-AUC score, but it produced many false positives. This means it marked many normal transactions as fraud.

5.2 Random Forest Model

Random Forest was implemented as the main model. It combines many decision trees and can learn nonlinear patterns in the data.

Table 6: Random Forest configuration

Parameter	Value
n_estimators	100
max_depth	10
class_weight	balanced
random_state	42
n_jobs	-1

Random Forest was also useful because it provides feature importance values. These values help explain which features were most important for fraud detection.

6 Midterm Model Comparison

During the midterm phase, Logistic Regression and Random Forest were compared. This helped decide which model should be used as the final model.

Table 7: Midterm comparison of Logistic Regression and Random Forest

Metric	Logistic Regression	Random Forest
ROC-AUC	0.9908	0.9993
True Positives	1,579	1,624
False Positives	63,657	12,907
False Negatives	64	19
True Negatives	1,207,224	1,257,974

Random Forest performed better than Logistic Regression. It detected more fraud cases, missed fewer fraud cases, and reduced false positives strongly.

Model Selection

Random Forest was selected as the final model because it gave better fraud detection results and produced fewer false positives than Logistic Regression.

7 Final Model Selection

Random Forest was fixed as the final model for the final phase of the project.

The main reasons are:

- It detected more fraud cases than Logistic Regression.
- It missed fewer fraud cases.
- It reduced false positives significantly.
- It can learn nonlinear relationships.
- It provides feature importance, which makes the model easier to explain.

No extra advanced models were added in the final phase. The focus was to make the Random Forest solution complete, clear, and well evaluated.

8 Final Random Forest Results

8.1 Confusion Matrix

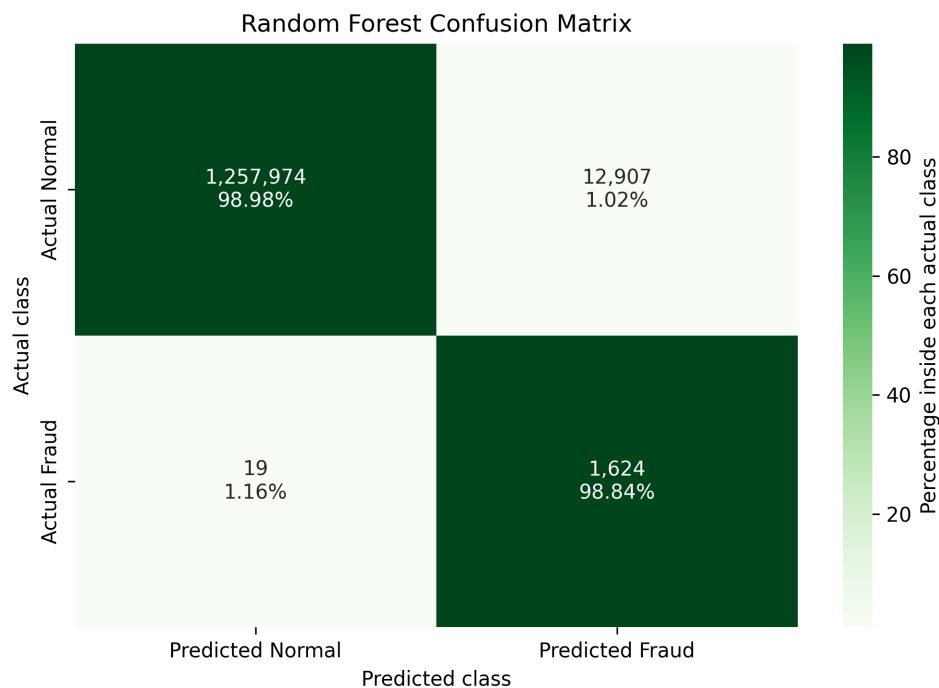


Figure 4: Final Random Forest confusion matrix.

Table 8: Random Forest confusion matrix values

Actual class	Predicted Normal	Predicted Fraud
Normal	1,257,974	12,907
Fraud	19	1,624

The model correctly detected 1,624 fraud cases and missed only 19 fraud cases. This is a strong result because fraud detection should minimize missed fraud transactions.

8.2 Final Evaluation Metrics

Table 9: Final Random Forest evaluation metrics

Metric	Value
Accuracy	0.9898
Balanced Accuracy	0.9891
Precision	0.1118
Recall	0.9884
Specificity	0.9898
F1-score	0.2008
ROC-AUC	0.9993
False Positive Rate	0.0102
False Negative Rate	0.0116
True Positives	1,624
False Positives	12,907
False Negatives	19
True Negatives	1,257,974

Recall is very high, which means the model detects almost all fraud cases. Precision is lower because the dataset is highly imbalanced. Since fraud is very rare, even a small false positive rate can create many false alarms.

8.3 ROC Curve

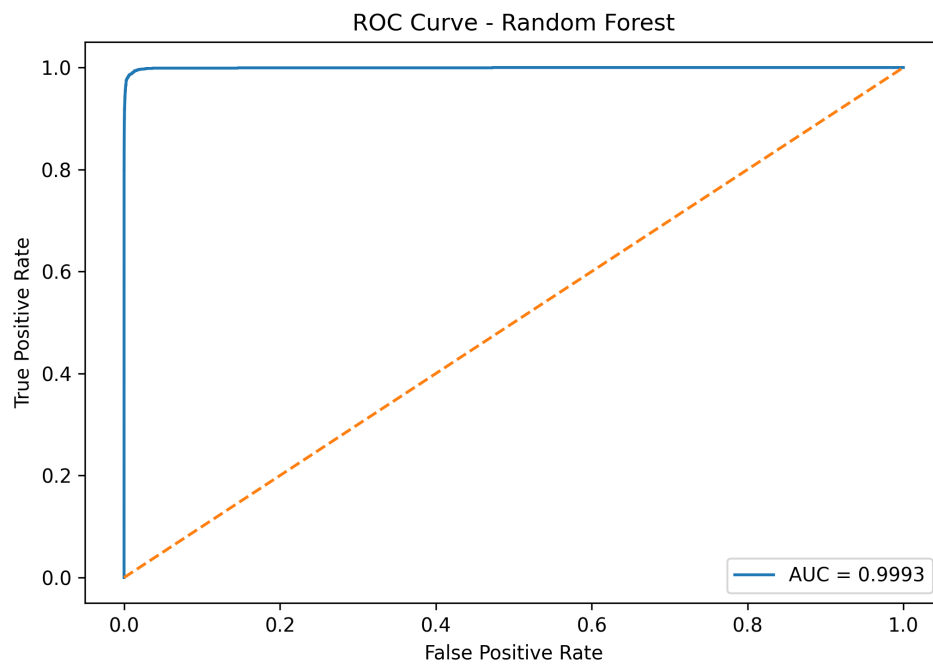


Figure 5: ROC curve for the final Random Forest model.

The ROC-AUC score is 0.9993. This shows that the model is very good at separating normal and fraudulent transactions.

8.4 Precision-Recall Curve

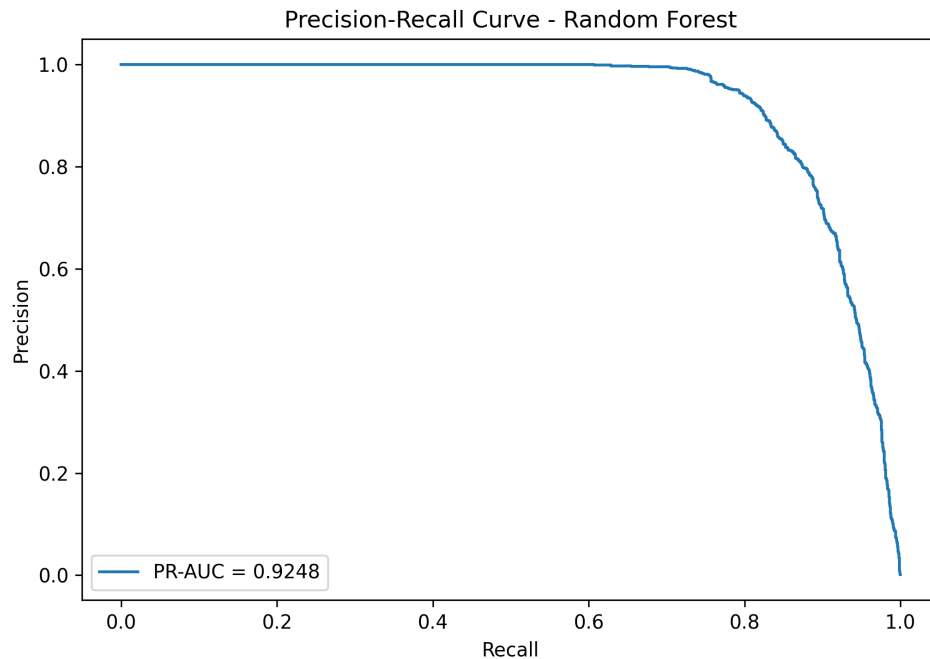


Figure 6: Precision-Recall curve for the final Random Forest model.

The Precision-Recall curve is useful because the dataset is highly imbalanced. It shows the trade-off between detecting fraud cases and reducing false alerts.

9 Feature Importance Analysis

Random Forest gives feature importance values. These values show which features helped the model most.

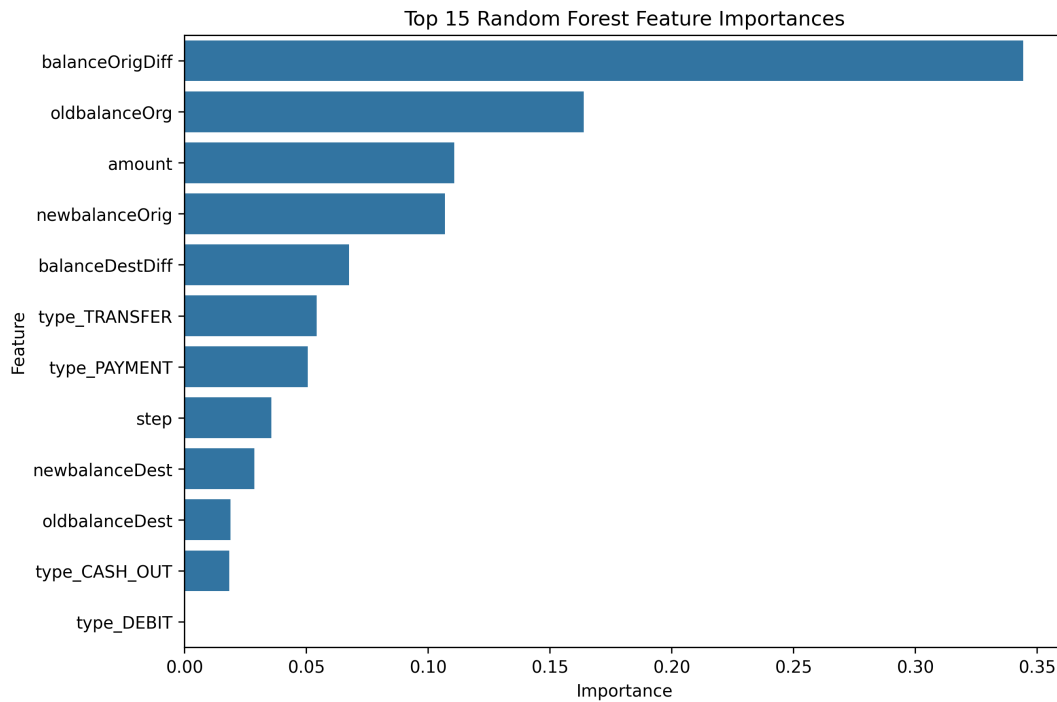


Figure 7: Random Forest feature importance.

The most important feature is `balanceOrigDiff`. This means the change in the sender account balance is very useful for detecting fraud.

9.1 Main Observations from Feature Importance

- `balanceOrigDiff` is the most important feature.
- Sender balance features are more important than receiver balance features.
- `amount` is important, but it is not enough alone.
- Transaction type helps because fraud mainly appears in `TRANSFER` and `CASH_OUT`.
- The engineered balance features improved the model explanation.

10 Discussion

The final results show that Random Forest is suitable for this fraud detection task. The model achieved high recall and very high ROC-AUC. This means it can detect most fraud cases and separate fraud transactions from normal transactions well.

10.1 Important Observations

- The model detected 1,624 out of 1,643 fraud cases in the test set.
- Only 19 fraud cases were missed.
- The number of false positives was much lower than in Logistic Regression.
- Balance-related features were the most important features.
- Log-scale plots were needed because the dataset is highly imbalanced.

10.2 Why Accuracy Alone Is Not Enough

Accuracy can be misleading in fraud detection. Since most transactions are normal, a model can get high accuracy by predicting almost everything as normal. For this reason, recall, precision, F1-score, ROC-AUC, and PR-AUC are more useful.

10.3 Business Meaning

In fraud detection, false negatives and false positives have different effects.

Table 10: Meaning of prediction errors

Error Type	Meaning
False Negative	A fraud transaction is predicted as normal. This is dangerous because the fraud is missed.
False Positive	A normal transaction is predicted as fraud. This may inconvenience the customer.

For this project, reducing false negatives is very important. The final Random Forest model performs well because it misses very few fraud transactions.

11 Handling Class Imbalance

Class imbalance was one of the main challenges in this project. Fraud transactions represent only 0.129% of the dataset.

11.1 Class Weighting

The model used:

```
class_weight = 'balanced'
```

This gives more importance to the minority class during training. It helps the model learn fraud patterns better.

11.2 Stratified Sampling

The train-test split used stratified sampling. This keeps the fraud ratio similar in both training and test sets. This makes the evaluation more reliable.

12 Implementation Details

The final project was implemented in Python.

12.1 Libraries Used

Table 11: Python libraries used

Library	Purpose
pandas	Data loading and data processing.
numpy	Numerical operations.
scikit-learn	Model training and evaluation.
matplotlib	Plot creation.
seaborn	Statistical visualizations.
joblib	Saving the trained model.

12.2 Code Structure

The final code is organized into small functions. Each function has one clear purpose, which makes the project easier to read, run, and explain during the presentation.

Data Loading and Setup

- `create_output_folders()`
Creates the folders needed for saving results and the trained model.
- `clean_old_results()`
Removes old result files so the folder only contains the newest outputs.
- `load_dataset()`
Loads the PaySim CSV dataset.
- `print_basic_information()`
Prints basic dataset information such as shape, missing values, and class distribution.

Exploratory Data Analysis

- `create_eda_plots()`
Runs all exploratory data analysis plots.
- `fraud_distribution_log_scale()`
Creates a readable fraud distribution plot using log scale.
- `transaction_type_analysis()`
Creates a transaction type vs fraud plot and saves a summary table.
- `amount_distribution()`
Creates the transaction amount distribution plot using log scale.

Data Preparation and Model Training

- `prepare_data()`
Performs feature engineering, removes unnecessary columns, applies one-hot encoding, and splits the data.
- `train_random_forest()`
Trains the final Random Forest model.

Model Evaluation and Saving

- `evaluate_random_forest()`
Calculates the final evaluation metrics and saves metric tables.
- `create_evaluation_plots()`
Creates the final model evaluation plots.
- `main()`
Runs the full project pipeline from loading data to saving the final model.

12.3 How to Run the Code

Place the PaySim CSV file in the same folder as the Python file. Then run:

```
pip install pandas numpy scikit-learn matplotlib seaborn joblib
python Project_final.py
```

The script creates two folders automatically:

- `results/`: contains plots and metric tables.
- `models/`: contains the saved Random Forest model.

12.4 Generated Output Files

Table 12: Main generated output files

File	Purpose
<code>fraud_distribution_log_scale.png</code>	Shows the class imbalance clearly.
<code>transaction_type_vs_fraud_log_scale.png</code>	Shows fraud distribution by transaction type.
<code>amount_distribution.png</code>	Shows transaction amount distribution.
<code>random_forest_confusion_matrix.png</code>	Shows correct and incorrect predictions.
<code>random_forest_roc_curve.png</code>	Shows ROC-AUC performance.
<code>random_forest_precision_recall_curve.png</code>	Shows the precision-recall trade-off.
<code>random_forest_feature_importance.png</code>	Shows the most important model features.
<code>random_forest_metrics.csv</code>	Stores final metric values.
<code>random_forest_model.pkl</code>	Stores the saved trained model.

13 Challenges Faced

The main challenges were:

- The dataset was highly imbalanced.

- Normal count plots did not show fraud cases clearly.
- Logistic Regression created many false positives.
- Balance columns needed feature engineering to become more useful.
- The dataset was large, so some plots needed log scale or sampling.

These issues were handled using class weighting, stratified sampling, improved plots, and balance-difference features.

14 Conclusion

This project developed a machine learning solution for fraud detection using the PaySim dataset. The project started with data exploration and preprocessing. Then Logistic Regression and Random Forest were implemented and compared during the midterm phase.

The comparison showed that Random Forest was the better model. It achieved a ROC-AUC score of 0.9993, detected 1,624 fraud cases, and missed only 19 fraud cases in the test set. It also produced far fewer false positives than Logistic Regression.

The final solution selected Random Forest as the main model. The final pipeline includes feature engineering, one-hot encoding, stratified splitting, class imbalance handling, model evaluation, result visualization, and model saving.

The feature importance results showed that balance-related features, especially `balanceOrigDiff`, were the strongest indicators of fraud. Overall, the project shows that machine learning can be used effectively to detect suspicious financial transactions.

15 References

References

- [1] PaySim Dataset, <https://www.kaggle.com/datasets/ealaxi/paysim1>
- [2] Scikit-learn Documentation, <https://scikit-learn.org/>
- [3] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.